KarlOS

USER MANUAL

BY:

ZENADINE ZIDANE PABLO

RITCHIE CONRAD HOO

KARL FREDERICK SITJAR

JHUNCEL LORIOSA

DANICA GRACE PERJES

KARLOS

USER MANUAL

1. INTRODUCTION

KarlOS is a Python-based Mini Operating System simulation designed to demonstrate basic operating system concepts through a command-line interface.

The system allows users to interact with a simulated file system, manage processes, monitor memory usage, and execute commands through a terminal. It also provides basic system information, command history, and error messages to help users understand how the system operates.

KarlOS is an educational simulation and does not replace an actual operating system such as Windows or Linux.

2. SYSTEM REQUIREMENTS

The following are required to run KarlOS:

- A computer running Windows, Linux, macOS, or another operating system that supports Python.
- Python installed on the computer.
- The complete KarlOS project folder, including all required source files.
- A terminal or command prompt.
- Sufficient free storage space to extract and run the project.

The project uses Python's standard libraries and does not appear to require additional third-party packages.

3. INSTALLATION

Follow these steps to prepare KarlOS for use.

1. Obtain the KarlOS project ZIP file from the project repository or group.
2. Locate the downloaded file named "Karl OS.zip".
3. Right-click the ZIP file and select "Extract All" or use an archive extraction program.
4. Choose a destination folder where the project will be stored.
5. Open the extracted KarlOS folder.
6. Confirm that the main program file, main.py, and the required Python modules are present.
7. Ensure that Python is installed on the computer.

To verify the Python installation, open Command Prompt or a terminal and enter:

python --version

If the command is not recognized on Windows, try:

py --version

If Python is not installed, download and install it from the official Python website:

https://www.python.org/downloads/

4. HOW TO START KARLOS

Follow these steps to start the Mini OS.

1. Open the folder containing the extracted KarlOS project.
2. Open Command Prompt or a terminal.
3. Navigate to the project folder using the cd command. For example:

cd "C:\Users\User\Documents\Karl OS"

Replace the example path with the actual location of your project folder.

4. Start the program by entering:

python main.py

If necessary, use the following command on Windows:

py main.py

5. Wait for the startup screen to appear.
6. When the welcome message is displayed, the system is ready to receive commands.

The terminal will display a prompt similar to:

KarlitoBoy@minios:/$

The prompt identifies the current simulated user and working directory. Enter commands after the dollar sign and press Enter to execute them.

To view the available commands, type:

help

5. LOGIN INSTRUCTIONS

The current version of KarlOS does not require a username or password to log in. It starts directly at the command prompt and uses a preset simulated user.

The default user displayed in the prompt is:

KarlitoBoy

To check the current user, enter: whoami

The system displays the active simulated username.

To begin using KarlOS, type any supported command after the prompt and press Enter.

To end the session, enter:

exit

Note: The simulated user is part of the MiniOS program. It does not represent a Windows or Linux account and does not provide password-based security.

6. COMMAND LIST

Commands are entered at the KarlOS terminal prompt. After typing a command, press Enter to execute it.

For commands that require arguments, replace the placeholders with the appropriate values. For example, in "mkdir <name>", replace <name> with the name of the directory you want to create.

6.1 GENERAL AND SYSTEM COMMANDS

COMMAND: HELP

Purpose:
Displays the available commands and their descriptions. It can also provide information about a specific command.

Syntax:
help [command]

Example:
help

Example for a specific command:
help mkdir

COMMAND: WHOAMI

Purpose:
Displays the name of the current simulated user.

Syntax:
whoami

Example:
whoami

COMMAND: DATE

Purpose:
Displays the current date and time using the computer's clock.

Syntax:
date

Example:
date

COMMAND: UPTIME

Purpose:
Displays how long KarlOS has been running during the current session.

Syntax:
uptime

Example:
uptime

COMMAND: SYSTEMINFO

Purpose:
Displays system information configured in the program, such as the operating system name, version, architecture, CPU, memory, and user information.

Syntax:
systeminfo

Example:
systeminfo

COMMAND: CLEAR

Purpose:
Clears the terminal screen to make the command-line interface easier to read.

Syntax:
clear

Example:
clear

COMMAND: HISTORY

Purpose:
Displays the commands entered during the current session. The -c option is intended to clear the command history.

Syntax:
history [-c]

Example:
history

Example to clear the history:
history -c

COMMAND: EXIT

Purpose:
Ends the KarlOS command loop and closes the current MiniOS session.

Syntax:
exit

Example:
exit

Note: The commands quit and logout are intended as alternatives to exit, but their alias configuration may need correction in the current source code.

6.2 FILE MANAGEMENT COMMANDS

KarlOS includes a simulated file system that allows users to create directories and files, view file contents, and navigate between directories.

COMMAND: MKDIR

Purpose:
Creates one or more directories in the current working directory.

Syntax:
mkdir <name> [name...]

Example:
mkdir Documents

Example to create multiple directories:
mkdir Documents Pictures Music

COMMAND: LS

Purpose:
Displays the files and directories available in the current directory or in a specified path.

Syntax:
ls [path]

Example:
ls

Example to view another directory:
ls Documents

Directories may be displayed with a slash after their names.

COMMAND: CD

Purpose:
Changes the current working directory to another directory.

Syntax:
cd <path>

Example:
cd Documents

To return to the parent directory:
cd ..

COMMAND: PWD

Purpose:
Displays the full path of the current working directory in the simulated file system.

Syntax:
pwd

Example:
pwd

COMMAND: CREATE

Purpose:
Creates a new file and can optionally include initial text content.

Syntax:
create <file> [text...]

Example:
create notes.txt Hello KarlOS

The file contains the text provided when the command is executed.

Another example:
create notes.txt "This is my first file."

COMMAND: CAT

Purpose:
Displays the text stored inside a file.

Syntax:
cat <file>

Example:
cat notes.txt

COMMAND: DEL

Purpose:
Intended to delete a file or an empty directory.

Syntax:
del <name> [name...]

Example:
del notes.txt

Important: The current source code contains a command-alias issue affecting del. The alias redirects the command to rm, which is not registered as a supported command. Consequently, del may fail until the alias is corrected in shell.py.

6.3 PROCESS MANAGEMENT COMMANDS

KarlOS provides a simulated process manager. Each process receives a process identification number (PID), allowing users to view and manage simulated processes.

COMMAND: PROCESS

Purpose:
Displays the simulated running processes, including information such as their PID, user, state, running time, and process name.

Syntax:
process

Example:
process

When KarlOS starts, it automatically creates an initial process named init with PID 1.

COMMAND: RUN

Purpose:
Starts a new simulated process and displays its assigned PID.

Syntax:
run <name>

Example:
run Calculator

Example output:
Started 'Calculator' with PID 2

The PID may differ depending on which processes were started earlier in the session.

COMMAND: KILL

Purpose:
Terminates a simulated process using its PID. Memory associated with the terminated process is also released.

Syntax:
kill <pid>

Example:
kill 2

The specified PID must belong to an existing process. The initial init process with PID 1 is protected and cannot be terminated.

6.4 MEMORY MANAGEMENT COMMANDS

KarlOS provides a simulated memory manager that tracks memory allocated to processes. These commands do not control the computer's actual RAM.

COMMAND: MEMORY

Purpose:
Displays the total, used, and free simulated memory. It can also show memory allocations associated with individual processes.

Syntax:
memory

Example:
memory

COMMAND: ALLOCATE

Purpose:
Allocates a specified amount of simulated memory, measured in megabytes (MB), to an existing process.

Syntax:
allocate <pid> <mb>

Example:
allocate 2 100

This example requests 100 MB for the process with PID 2.

The process must exist, the requested amount must be positive, and sufficient simulated memory must be available.

COMMAND: FREE

Purpose:
Releases all simulated memory currently allocated to a specified process.

Syntax:
free <pid>

Example:
free 2

The system displays the amount of memory released. The process must have an existing memory allocation for the operation to succeed.

7. BASIC OPERATING PROCEDURES

7.1 CREATING AND READING A FILE

This procedure demonstrates basic file management.

Step 1: Create a directory.

mkdir Documents

Step 2: Enter the directory.

cd Documents

Step 3: Create a file containing text.

create notes.txt "Welcome to KarlOS."

Step 4: Display the file contents.

cat notes.txt

Step 5: Return to the parent directory.

cd ..

Step 6: View the available files and directories.

ls

7.2 RUNNING AND MANAGING A PROCESS

This procedure demonstrates how to start a process, allocate memory, and terminate it.

Step 1: Start a simulated process.

run Calculator

Step 2: Display the process list.

process

Step 3: Identify the PID assigned to Calculator.

Step 4: Allocate 100 MB to the process. Replace the example PID with the actual PID shown by KarlOS.

allocate 2 100

Step 5: Check the memory status.

memory

Step 6: Release the allocated memory.

free 2

Step 7: Terminate the process when it is no longer needed.

kill 2

Always use the actual PID displayed by KarlOS rather than assuming the process will receive PID 2.

8. ERROR HANDLING

KarlOS displays error messages when a command is invalid or an operation cannot be completed.

Common situations include:

- Command not found: The entered command is not recognized.
- Missing arguments: The command requires additional information.
- File or directory not found: The specified file or path does not exist.
- Directory already exists: A directory with the same name is already present.
- Directory not empty: A directory containing files or other directories cannot be deleted.
- Invalid PID: The specified process does not exist.
- Insufficient memory: The requested allocation exceeds the available simulated memory.
- Invalid memory amount: The allocation amount must be a positive number.
- Invalid command syntax: The command or its arguments were entered in an incorrect format.

When an error occurs, read the displayed message, check the command spelling and required arguments, and try again.

9. TROUBLESHOOTING

PROBLEM: KARLOS DOES NOT START

Solution:
Confirm that Python is installed, that the terminal is in the correct project folder, and that main.py and the required Python modules are present.

PROBLEM: A COMMAND IS NOT RECOGNIZED

Solution:
Enter help to view the available commands. Check the spelling and required arguments.

PROBLEM: A FILE CANNOT BE FOUND

Solution:
Use pwd to check the current directory and ls to display its contents. Confirm that the file name and path are correct.

PROBLEM: A PROCESS CANNOT BE TERMINATED

Solution:
Use process to verify that the PID exists. Remember that the initial init process with PID 1 is protected.

PROBLEM: MEMORY ALLOCATION FAILS

Solution:
Use memory to check the available simulated memory. Verify that the PID exists and that the requested allocation is a valid positive number.

PROBLEM: THE DEL COMMAND DOES NOT WORK

Solution:
The current command-alias configuration redirects del to an unregistered rm command. Correct the alias in shell.py before using the delete command.

PROBLEM: THE QUIT OR LOGOUT COMMAND DOES NOT WORK

Solution:
These aliases may have the same configuration issue. Use exit to shut down KarlOS.

PROBLEM: CHANGES TO FILES DISAPPEAR AFTER EXITING

Solution:
This is expected behavior in the current implementation. The simulated file system is stored in memory and does not preserve its contents between sessions.

10. IMPORTANT NOTES AND LIMITATIONS

- KarlOS is an educational MiniOS simulation and is not a complete operating system.
- The simulated file system is stored in memory and is not saved to the computer's actual storage.
- Files and directories created during a session will be lost when KarlOS exits.
- Processes are simulated and do not launch actual applications on the computer.
- Memory allocation is simulated and does not reserve the computer's real RAM.
- System information is configured in the source code and should not automatically be assumed to represent the actual computer hardware.
- The command history exists only during the current session.
- The current implementation does not provide a username/password login system.
- The current implementation does not provide persistent storage or a dedicated file-editing command.
- Some command aliases may require correction before they can be used successfully.
- Commands should be entered according to their documented syntax.

11. CONCLUSION

KarlOS demonstrates fundamental operating system concepts through a Python-based command-line environment. Its file management, process management, memory allocation, system information, and command handling features provide a practical introduction to how operating systems organize and manage resources.

Users can explore these features by entering commands in the terminal and observing the results of each operation. This MiniOS serves as an educational project for understanding basic operating system behavior, command-line interaction, and resource management.